National Cyber Security Centre

NCSC Vulnerability Report

NCSC Reference: 23011101
February 2023
© Crown Copyright 2023

This information is exempt under the Freedom of Information Act 2000 (FOIA) and may be exempt under other UK information legislation. Refer any FOIA queries to ncscinfoleg@ncsc.gov.uk

About this document

This document details a vulnerability identified by the National Cyber Security Centre (NCSC). These vulnerabilities reduce the security of the application.

Bug Bounty Payment

If this vulnerability is eligible for a Bug Bounty payment, we ask that the money be donated directly to NSPCC (Registered Charity Number: 216401, *https://www.nspcc.org.uk*).
Please contact the NCSC mailbox to inform us of the donation amount and the donation date.

NCSC Contact Information

The vulnerability disclosure mailbox is 'security@ncsc.gov.uk'. Please contact us for our PGP key.

Crediting NCSC

NCSC would appreciate appropriate credit as 'The UK's National Cyber Security Centre (NCSC)' in any advisories which you may publish about this issue.

Verification, Resolution and Release

Please inform NCSC via the 'security@ncsc.gov.uk' mailbox, quoting the NCSC Reference above, should you:

- confirm that this is a security issue allocate
- the issue a CVE identifier determine a date to
- release a patch determine a date to publish
- advisories

NCSC Disclosure Policy

NCSC has adopted the ISO 29147 approach to vulnerability disclosure and, as such, follows a coordinated disclosure approach with affected parties. We have never publicly disclosed a vulnerability prior to a fix being made available.

NCSC recognises that vendors need a reasonable amount of time to mitigate a vulnerability, for example, to understand the impact to customers, to triage against other vulnerabilities, to implement a fix in coordination with others, and to make that fix available to its customers. As this will vary based on the exact situation NCSC does not define a set time frame in which a fix must be made available, and we are happy to discuss the circumstances of any particular disclosure.

If NCSC believes a vendor is not making appropriate progress with vulnerability resolution, we may, after discussion with the vendor, choose to share the details appropriately (for example, with service providers and our customers) to ensure that we provide appropriate mitigation of the threat to the UK and to UK interests.

Disclaimer

Any NCSC findings and recommendations made have not been provided with the intention of avoiding all risks, and following the recommendations will not remove all such risk. Ownership of information risks remains with the relevant system owner at all times.

Summary

4 UAF vulnerabilities have been found in PDFium in Google Chrome 92, resulting in member setting and memory reads.

Details

The ‘m_Maps’ member of the ‘CFFL_FormFiller’ class stores pointers to ‘CPWL_Wnd’ objects. These pointers are retrieved through the ‘CFFL_ FormFiller::GetPWLWindow()’ method at various points in code. In some cases, it is possible to free the ‘CPWL_Wnd’ object pointed at between the time it is retrieved from the map and the time it is used, resulting in a UAF vulnerability. This is only possible with XFA enabled.

There are four potential instances of this occurring. Two of these relate to checkboxes, one to a listbox, and one to a textfield.

In all four cases, call paths through ‘CPDFSDK_Widget::GetMixXFAWidget()’ using careful naming of a widget result in JavaScript execution. A widget named ‘((\(eval\(‘d\(\)’\)!=0\)))’ will result in the JavaScript function ‘d()’ being executed. This function is defined in the XFA sub-form:

```
<subform>
      <field name="ChoiceList1"/>
      <event activity="initialize">
            <script contentType="application/x-javascript">
                  d = function() {
                  [2] app.trigger();
                        }
            </script>
      </event>
</subform>
```

At this point, forcing a call to ‘CFFL_ FormFiller::DestroyPWLWindow()’ clears the ‘m_Maps’ member and frees the ‘CPWL_Wnd’ objects. This can be achieved by changing a property of a field and setting focus to it. However, this is not possible from the XFA context, as a check exists in ‘CJS_App::get_active_docs()’ at [3] which will return a failure:

```
CJS_Result CJS_App::get_active_docs(CJS_Runtime* pRuntime) {
v8::Local<v8::Object> pObj = pRuntime->GetThisObj();
[3] auto pJSDocument = JSGetObject<CJS_Document>(pObj);
if (!pJSDocument)
      return CJS_Result::Failure(JSMessage::kObjectTypeError);
```

To get around this, a function is defined in the non-XFA form of the PDF [4] and called from [2] instead:

```
<</S/JavaScript/JS(
      app.i = 0;
[4]   app.trigger = function () {
            if (app.i == 5)
            {
                  app.activeDocs[0].getField("select").setFocus();
                  app.activeDocs[0].getField("(\(eval\('d\(\)'\)!=0\))").borderStyle =
            "dashed";
                  app.activeDocs[0].getField("(\(eval\('d\(\)'\)!=0\))").setFocus();
            }
            app.i++;
      }
```

```
)/Type/Action>>
```

1. UAF for a CheckBox pointer

```
bool CFFL_CheckBox::OnChar(CPDFSDK_Annot* pAnnot,
uint32_t nChar,
uint32_t nFlags) {
...
[0] CPWL_CheckBox* pWnd = GetCheckBox(pPageView, true);
if (pWnd && !pWnd->IsReadOnly()) {
CPDFSDK_Widget* pWidget = ToCPDFSDKWidget(pAnnot);
[1] pWnd->SetCheck(!pWidget->IsChecked());
}
...
```

At [0] the pointer is retrieved from the map.
At [1] in the function 'IsChecked' it is possible to execute JavaScript.
At [1] the widget pWnd was pointing to has been destroyed by the time 'SetCheck' is called, resulting in UAF.

2. UAF for a second CheckBox pointer

```
bool CFFL_CheckBox::OnLButtonUp(CPDFSDK_PageView* pPageView, CPDFSDK_Annot* pAnnot,
uint32_t nFlags,
const CFX_PointF& point) {
CFFL_Button::OnLButtonUp(pPageView, pAnnot, nFlags, point);
...
[0]   CPWL_CheckBox* pWnd = CreateOrUpdatePWLCheckBox (pPageView, true);
if (pWnd) {
      CPDFSDK_Widget* pWidget = ToCPDFSDKWidget(pAnnot);
[1]         pWnd->SetCheck(!pWidget->IsChecked());
      }
return CommitData(pPageView, nFlags);
}
```

At [0] the pointer is retrieved from the map.
At [1] in the function 'IsChecked' it is possible to execute JavaScript.
At [1] the widget pWnd was pointing to has been destroyed by the time 'SetCheck' is called, resulting in UAF.

3. UAF for a ListBox pointer

```
void CFFL_ListBox::SaveData(CPDFSDK_PageView* pPageView) {
[0] CPWL_ListBox* pListBox = GetListBox(pPageView);
if (!pListBox)
return;
int32_t nNewTopIndex = pListBox->GetTopVisibleIndex();
[1] m_pWidget->ClearSelection();
if (m_pWidget->GetFieldFlags() & pdfium::form_flags::kChoiceMultiSelect) {
[2] for (int32_t i = 0, sz = pListBox->GetCount(); i < sz; i++) {
      if (pListBox->IsItemSelected(i))
            m_pWidget->SetOptionSelection(i);
}
} else {
m_pWidget->SetOptionSelection(pListBox->GetCurSel());
}
...
```

At [0] the pointer is retrieved from the map ('GetListBox' makes the call to 'GetPWLWindow').
At [1] in the function 'ClearSelection' it is possible to execute JavaScript.

At [2] the widget ‘pListBox’ was pointing to has been destroyed by the time ‘GetCount’ is called, resulting in UAF.

4. UAF for a CPWL_Edit pointer

```
void CFFL_TextField::SaveData(CPDFSDK_PageView* pPageView) {
[0]   CPWL_Edit* pWnd = GetPWLEdit (pPageView, false);
if (!pWnd)
      return;
[1]   WideString sOldValue = m_pWidget->GetValue();
[2]   WideString sNewValue = pWnd->GetText();
```

At [0] the pointer is retrieved from the map (‘GetPWLEdit’ makes the call to ‘GetPWLWindow’).
At [1] in the function `GetValue` it's possible to execute JavaScript.
At [2] the widget pWnd was pointing to has been destroyed by the time ‘GetText’ is called, resulting in UAF.

Mitigation

This could be prevented by avoiding unnecessary raw pointers. XFA should also be disabled by default.